

Question 1 virtual address of single variable

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 practical.c  
#include <stdio.h>  
  
int main()  
{  
    int x = 10;  
  
    printf("Value of x = %d\n", x);  
    printf("Virtual address of x = %p\n", (void *)&x);  
  
    return 0;  
}
```

```
sri_dhruthi@Dhruthi:~$ nano practical.c  
sri_dhruthi@Dhruthi:~$ gcc practical.c -o practical  
sri_dhruthi@Dhruthi:~$ ./practical  
Value of x = 10  
Virtual address of x = 0x7ffc4bbf88c4
```

Question 2

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 practical2.c *  
#include <stdio.h>  
#include <stdlib.h>  
  
int global = 10;  
int bss_variable;  
  
int main()  
{  
    int local = 20;  
  
    int *heap = malloc(sizeof(int));  
    *heap = 30;  
  
    printf("Code address : %p\n", (void *)main);  
    printf("Data address : %p\n", (void *)&global);  
    printf("BSS address : %p\n", (void *)&bss_variable);  
    printf("Heap address : %p\n", (void *)heap);  
    printf("Stack address : %p\n", (void *)&local);  
  
    free(heap);  
  
    return 0;  
}
```

```

sri_dhruthi@Dhruthi:~$ nano practical2.c
sri_dhruthi@Dhruthi:~$ gcc practical2.c -o practical2
sri_dhruthi@Dhruthi:~$ ./practical2
Code address : 0x63ee8addc1a9
Data address : 0x63ee8addf010
BSS address : 0x63ee8addf018
Heap address : 0x63ee8c4ac010
Stack address : 0x7ffd9990e50c
sri_dhruthi@Dhruthi:~$

```

Question 3 Demand paging

```

GNU nano 8.7.1 demandpaging.c *
#include <stdio.h>

int main()
{
    int page_table[4] = {1, -1, 1, 1};
    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page < 0 || page > 3)
    {
        printf("Invalid page number!\n");
    }
    else if (page_table[page] == -1)
    {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else
    {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}

```

```

sri_dhruthi@Dhruthi:~$ nano demandpaging.c
sri_dhruthi@Dhruthi:~$ gcc demandpaging.c -o demandpaging
sri_dhruthi@Dhruthi:~$ ./demandpaging
Enter page number (0-3): 0
Page is already in memory.
Physical frame = 1
sri_dhruthi@Dhruthi:~$ ./demandpaging
Enter page number (0-3): 1
Page Fault!
Page must be loaded into memory.

```

Question 4 Address translating

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1      adresstranslating.c *  
#include <stdio.h>  
  
int main()  
{  
    int page_table[] = {3, 5, 1, 7};  
    int page_size = 100;  
  
    int virtual_address = 250;  
  
    int page = virtual_address / page_size;  
    int offset = virtual_address % page_size;  
  
    int frame = page_table[page];  
  
    int physical_address = frame * page_size + offset;  
  
    printf("Virtual Address   = %d\n", virtual_address);  
    printf("Page Number      = %d\n", page);  
    printf("Offset           = %d\n", offset);  
    printf("Frame Number     = %d\n", frame);  
    printf("Physical Address = %d\n", physical_address);  
  
    return 0;  
}
```

```
sri_dhruthi@Dhruthi:~$ nano adresstranslating.c  
sri_dhruthi@Dhruthi:~$ gcc adresstranslating.c -o adresstranslating  
sri_dhruthi@Dhruthi:~$ ./adresstranslating  
Virtual Address   = 250  
Page Number      = 2  
Offset           = 50  
Frame Number     = 1  
Physical Address = 150
```

Question 5 dynamic memory

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 dynamicmemory  
#include <stdio.h>  
#include <stdlib.h>  
  
int main()  
{  
    int *p;  
  
    p = malloc(5 * sizeof(int));  
  
    if (p == NULL)  
    {  
        printf("Memory allocation failed\n");  
        return 1;  
    }  
  
    for (int i = 0; i < 5; i++)  
    {  
        p[i] = (i + 1) * 10;  
    }  
  
    for (int i = 0; i < 5; i++)  
    {  
        printf("%d ", p[i]);  
    }  
  
    printf("\n");  
  
    free(p);  
  
    return 0;  
}|  
  
sri_dhruthi@Dhruthi:~$ nano dynamicmemory.c  
sri_dhruthi@Dhruthi:~$ gcc dynamicmemory.c -o dynamicmemory  
sri_dhruthi@Dhruthi:~$ ./dynamicmemory  
10 20 30 40 50
```

Question 6 segmentation fault

```
GNU nano 8.7.1 nullpointe
#include <stdio.h>

int main()
{
    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

```
sri_dhruthi@Dhruthi:~$ nano nullpointer.c
sri_dhruthi@Dhruthi:~$ gcc nullpointer.c -o nullpointer
sri_dhruthi@Dhruthi:~$ ./nullpointer
Before accessing memory
Segmentation fault      ./nullpointer
sri_dhruthi@Dhruthi:~$
```